

Revisiting German NER with XLM-R Wearing a CRF Hat

Danny Rechitsky
Brandeis University
Computational Linguistics
drechit@gmail.com

1 Introduction

Conneau et al. (2020)’s XLM-R paper demonstrated that a single multilingual encoder, pre-trained on 2.5 TB of cleaned Common Crawl data across 100 languages without any explicit language identifiers, can match or exceed monolingual state-of-the-art models on a range of cross-lingual benchmarks. On the CoNLL-2003 (Tjong Kim Sang and De Meulder, 2003) and CoNLL-2002 (Tjong Kim Sang, 2002) NER tasks, however, XLM-R fell short of one notable competitor: Akbik et al. (2018), a BiLSTM-CRF tagger over Flair contextual string embeddings, which still held the state of the art on German. Conneau et al. attribute part of that gap to their decision not to put a CRF decoder on top of XLM-R, leaving open the question of whether such a head would close—or even overshoot—the gap.

This paper picks up that thread. We first replicate the XLM-R Base fine-tuning experiments on CoNLL-2003 EN and DE, then test whether adding a CRF head on top of XLM-R can push it past Akbik et al. (2018) on the German test set. Beyond the specific German NER setting, our results point to a broader observation: CRF heads on top of masked language models can recover span-level structure that the encoder’s token-level predictions miss, at least when the encoder has not already absorbed that structure through end-to-end fine-tuning. Further below we present the data and modeling setup (Sections 3 and 4). We also report the replication results, which exceed the DE-finetuned model in Conneau et al. (2020) by 1.29% F1. Moreover, jointly training an XLM-R+CRF model outperforms the baseline of a DE-finetuned encoder-frozen XLM-R model by 5.52% F1 (Section 5).

2 Related Work

The push for a single multilingual encoder that transfers across languages predates XLM-R. Multi-

lingual BERT (mBERT) (Devlin et al., 2019) was trained jointly on Wikipedia in 104 languages with a single shared subword vocabulary and demonstrated surprisingly strong zero-shot cross-lingual transfer despite having no explicit cross-lingual supervision. Conneau and Lample (2019) extended this idea with XLM, which added a translation language modeling (TLM) objective that uses parallel sentences to align representations across languages. Critically, XLM also added *language embeddings* to its input: each token carried an explicit signal of which language it came from. While this helped monolingual performance, it allowed the model to lean on language identity rather than learn truly language-agnostic representations, putting a soft ceiling on how language-invariant the resulting embeddings could be.

Conneau et al. (2020) pushed in the opposite direction with XLM-R. They dropped the language embeddings entirely, removed the TLM objective, and trained only with standard masked language modeling on a vastly larger corpus: 2.5 TB of cleaned Common Crawl across 100 languages, with a sampling distribution that upweights low-resource languages. The hypothesis was that, deprived of any explicit language signal, the model would be forced to learn truly cross-lingual representations from the distribution of multilingual text alone. The empirical payoff was substantial: XLM-R matched or exceeded the prior state of the art on a wide range of cross-lingual benchmarks, and remained the strongest published baseline on cross-lingual NER for some time. They evaluated XLM-R on the NER shared tasks from CoNLL-2002 (Tjong Kim Sang, 2002) (Spanish, Dutch) and CoNLL-2003 (Tjong Kim Sang and De Meulder, 2003) (English, German): when fine-tuned on English training data and evaluated zero-shot on Spanish and Dutch, XLM-R exceeded the prior state of the art, but it transferred poorly to German under the same setup. Only when fine-tuned

directly on German training data did XLM-R approach state-of-the-art performance on the German test set, where it was still outstripped by [Akbik et al. \(2018\)](#), a BiLSTM-CRF tagger over contextual string embeddings. [Conneau et al. \(2020\)](#) themselves attribute part of this gap to their not having implemented a CRF decoder on top of XLM-R, which directly motivates the experiments in Section 5.

The closest precedent for that setup is [Souza et al. \(2019\)](#), who reported that BERT + CRF outperforms the prior BiLSTM-CRF state of the art on Portuguese NER, suggesting that the MLM + CRF recipe can succeed on non-English NER. Whether the same recipe carries over to German with XLM-R is the empirical question we take up below.

3 Data

We use the CoNLL-2003 NER shared task data ([Tjong Kim Sang and De Meulder, 2003](#)) for both English and German. The English portion is drawn from *Reuters Newswire* and the German portion from articles in the *Frankfurter Rundschau*. Each token in both languages corresponds to a named entity tag from the four-class set {PER, ORG, LOC, MISC}, where PER covers individual people, ORG covers companies and institutions, LOC covers countries, cities, and other named geographical entities, and MISC covers everything else that is named (nationalities, named events, languages, and the like). Tags are encoded with the BIO scheme: B-X marks the first token of an entity of type X, I-X marks a continuation token of an entity of type X, and O marks a non-entity token, for a total of nine possible labels. Documents are delimited by -DOCSTART- markers, which were originally introduced to support document-level features in shared task systems but are not used by our model, which operates at the token-level with consideration only for sentence boundaries but not document boundaries. Table 1 summarizes the splits we used after removing the -DOCSTART- lines and preserving sentence boundaries.

A practical caveat for the German half: the original CoNLL-2003 distribution shipped with broken UTF-8 encoding for the umlauts (ä, ö, ü) and ß characters that recur throughout German named entities. The issue is well documented and has been addressed in unofficial cleaned-up releases; we use a corrected version of the German data. This matters in practice because XLM-R’s Senten-

Lang	Split	Sents.	Tokens	Ments.
EN	train	14,041	203,621	23,499
EN	dev	3,250	51,362	5,942
EN	test	3,453	46,435	5,702
DE	train	12,152	206,931	10,008
DE	dev	2,867	51,444	3,931
DE	test	3,005	51,943	3,051

Table 1: CoNLL-2003 split statistics after preprocessing. Sentence counts exclude -DOCSTART- markers; mention counts are the number of B-tagged spans across the four entity types (PER, ORG, LOC, MISC).

cePiece tokenizer expects properly encoded UTF-8: broken encoding produces different (and noisier) subword segmentations of common entity tokens like “Müller” or “Köln,” which we believe partly accounts for the DE-only result reported in Section 5.

Beyond the totals in Table 1, the per-class distribution differs notably between the two languages. In the German training set, LOC is the dominant category (4,273 mentions, vs. 2,801 PER and 2,154 ORG), reflecting the heavy coverage of geographical and political entities in the *Frankfurter Rundschau* source articles; MISC is comparatively rare at 780 mentions. The English training set is more balanced, with PER, ORG, and LOC each in the 6,300–7,200 range and MISC at 3,438. These imbalances matter for evaluation: a model that does well on common classes can post a high micro-averaged F1 even with weaker performance on MISC.

Inputs are tokenized with the XLM-R SentencePiece tokenizer ([Conneau et al., 2020](#)). To align word-level NER labels with subword tokens, we assign each word’s BIO tag to its first subword and mask all continuation subwords, as well as the special tokens (<s>, </s>), with the index -100 so that they are ignored by the cross-entropy loss. Within each batch, sequences are dynamically padded to the length of the longest sentence in the batch.

For the EN+DE concatenated training set (last row of Table 2), we combine the English and German training data and shuffle the result with random seed 1; the development set used to monitor that run is the German dev split alone. All training runs reported in this paper use seed 1.

4 Models

4.1 Architecture and Training

We use XLM-R Base¹ (Conneau et al., 2020), a Transformer encoder with 12 layers, 12 attention heads, and a hidden dimension of 768. XLM-R was pretrained on 2.5 TB of cleaned Common Crawl data spanning 100 languages, with a sampling distribution that upweights low-resource languages, and without any language-specific embeddings or language identifiers; this encourages the model to learn implicitly multilingual representations.

For NER fine-tuning we follow the standard token classification setup: each token’s hidden state from the encoder is passed through a single linear projection (HuggingFace’s `AutoModelForTokenClassification` adds a $768 \rightarrow 9$ classifier layer with no intervening dropout or hidden layers) to produce per-token logits over the nine BIO labels. At inference time without a CRF, the predicted label for each token is the argmax over its logits.

We fine-tune XLM-R with AdamW at a learning rate of 2×10^{-5} and weight decay of 0.01 for 3 epochs, with random seed 1 for reproducibility.

A linear-chain CRF (Lafferty et al., 2001) models the joint distribution over a label sequence $y = (y_1, \dots, y_T)$ given an input sequence by combining two scoring components: *emission scores*—in our case, XLM-R’s per-token logits, which capture how well each token matches each label—and *transition scores*, a learned $|L| \times |L|$ matrix (where L is the label set) that captures how plausible each label-to-label transition is. The score of a candidate label sequence is the sum of its emission and transition scores, and the conditional probability is obtained by normalizing this score over all possible label sequences using the forward algorithm. At training time, the model maximizes the log-likelihood of the gold sequence; at inference time, the most likely label sequence is recovered with the Viterbi algorithm.

The transition matrix is the structural piece a CRF adds to a token-level classifier: it can encode hard constraints (e.g., I-PER cannot follow O or B-LOC under BIO) and learned soft preferences (e.g., B-LOC may be more likely than B-ORG after certain words). To accelerate learning of the BIO constraints, we initialize all impossible transitions: any I-X that would follow a label other than B-X

or I-X—to a very negative value ($-10,000$), so that the matrix begins from a state that already disallows ill-formed sequences and the optimizer only needs to refine the soft preferences.

For the CRF head, we combine a linear-chain CRF, implemented with the `pytorch-crf` library,² with XLM-R in three configurations. (i) In Table 3, “CRF on top of fine-tuned XLM-R” block, we take XLM-R already fine-tuned on DE and continue training only its top transformer layer along with the new CRF head. (ii) In Table 3, “CRF + XLM-R jointly trained” block, we jointly train the CRF together with XLM-R’s classification layer and its top $k \in \{1, 2, 3\}$ transformer layers, using AdamW with $\text{lr} = 10^{-4}$ for the XLM-R parameters, $\text{lr} = 10^{-3}$ for the CRF, and weight decay 0 for both groups. (iii) In Table 4, we run a head-to-head comparison in which only the top transformer layer is unfrozen (with the always-trainable classifier), once with and once without a jointly-trained CRF head. Both runs use AdamW at $\text{lr} = 2 \times 10^{-5}$ for the unfrozen transformer layer and the classifier (with weight decay 0.01 on the classifier); the CRF parameters are trained at $\text{lr} = 10^{-3}$.

In all CRF experiments, XLM-R’s encoder weights are frozen except for the top transformer layers explicitly listed as unfrozen; XLM-R’s classification layer—the linear projection from the encoder’s 768-dim hidden states to the 9 BIO label logits—is always trainable.

4.2 Hyperparameter Selection

The hyperparameters in this paper follow standard practice for transformer fine-tuning rather than results of a dedicated grid search. For XLM-R we use a small learning rate (2×10^{-5}) and a small weight decay (0.01): the pretrained weights already encode useful general-purpose representations, so large parameter updates risk catastrophically overwriting pretraining knowledge, and aggressive regularization would pull these weights toward zero and undo what pretraining provided. These values are widely adopted defaults for fine-tuning transformer encoders since the original BERT recipe.

The CRF head, by contrast, is randomly initialized at the start of training and has very few parameters: the transition matrix is 9×9 over the BIO label set, plus start- and end-of-sequence vectors. A higher learning rate (10^{-3}) lets these randomly-initialized parameters move quickly toward a useful

¹<https://huggingface.co/FacebookAI/xlm-roberta-base>

²<https://pytorch-crf.readthedocs.io/en/stable/>

configuration without being held back by the small-step regime appropriate for the encoder; the small parameter count also means each gradient update has an outsized effect on the loss, so a larger step size is well tolerated. Differential learning rates of this kind—small for the encoder, large for a randomly-initialized task head—are standard practice when adding a new head on top of a pretrained model.

All runs use 3 training epochs. This is a common choice for fine-tuning transformer encoders on CoNLL-sized datasets (on the order of 10^4 sentences): validation F1 typically plateaus by the third epoch in our runs, and pushing further begins to risk overfitting on a relatively small training set.

4.3 Evaluation

We report mention-level (span-level) F1 using the `segeval` library. Under this scoring scheme, a predicted entity is counted as correct only if both its span boundaries and its type label exactly match a gold mention; partial overlaps and type errors are counted as both a false positive (the prediction) and a false negative (the gold mention). This is a stricter criterion than token-level accuracy, which would count individual tokens correct even when the span as a whole is malformed.

F1 is micro-averaged across the four entity types, matching the CoNLL-2003 evaluation convention (Tjong Kim Sang and De Meulder, 2003) and the reporting setup of Conneau et al. (2020). Micro-averaging weights each entity mention equally, so frequent classes (LOC in our German data; PER, ORG, and LOC in English) contribute more to the headline number than rare classes (MISC). Macro averaging would treat each class equally regardless of frequency, which can be a more demanding criterion when class imbalance is severe; we report micro F1 for direct comparability with prior work on this dataset.

5 Results

5.1 Replication of XLM-R Fine-tuning

Table 2 reports four conditions, each isolating a different aspect of XLM-R’s NER capabilities. EN→EN and DE→DE are monolingual baselines, testing how well XLM-R adapts to a target language when given training data in that same language. EN→DE is the cross-lingual zero-shot transfer condition: the model is fine-tuned only on English data and then evaluated on German, iso-

lating how much of XLM-R’s pretraining transfers across languages without any target-language supervision. EN+DE is a multilingual concatenation condition, testing whether adding English training data to German training data helps or hurts DE performance.

The XLM-R Base reproduction tracks Conneau et al. (2020) closely on three of the four conditions. On EN→EN we reach 92.55 F1 versus their 92.25 — a +0.3 gap small enough to fall within seed and implementation variance.

The EN→DE zero-shot transfer row is the one notable deviation: we obtain 65.83 F1 against the reference 71.40. The 5.6-point gap is consistent with what is now well documented about cross-lingual zero-shot NER, where results vary substantially across random seeds and hyperparameter choices. With a single seed (`seed=1`) and a 3-epoch schedule, we land at the low end of what is plausible for this setting—a limitation of a single-run replication rather than a substantive challenge to the cross-lingual pretraining claim.

DE→DE comes in 1.3 points above the reference (87.10 vs 85.81). The most plausible explanation is the well-known encoding issue in the original CoNLL-2003 German release, whereas the German datasets used in our experiments had this issue corrected. With properly encoded UTF-8, in particular the umlauts that recur in German entities (e.g., “Müller”, “Köln”), XLM-R’s SentencePiece tokenizer produces clean subword segmentations.

EN+DE concatenated training degrades performance slightly relative to DE-only fine-tuning (86.51 vs 87.10 F1). With 14,041 EN sentences and 12,152 German sentences, the mixture tilts gradient updates toward English-specific patterns. A particularly pernicious source of confusion is that, in German, common nouns are also capitalized: a model that has internalized from English data a pattern like “capitalized $\Rightarrow$ likely named entity” will produce false positives on German common nouns. Since XLM-R Base’s multilingual pretraining already exposes the model to German, the additional English data offers no new structural information about German NER, it only adds a competing prior that hurts German-specific decisions.

5.2 Adding a CRF Head

The first attempt at improving the DE-fine-tuned XLM-R was to stack a CRF head on top of it and continue training only the top transformer layer alongside the new CRF (Table 3, “CRF on top of

Train	Test	This Work		Conneau et al.
		Dev	Test	Test
EN	EN	94.64	92.55	92.25
EN	DE	94.64	65.83	71.40
DE	DE	90.15	87.10	85.81
EN+DE	DE	90.56	86.51	—

Table 2: F1 on CoNLL-03 NER for XLM-R Base under four fine-tuning conditions, compared with [Conneau et al. \(2020\)](#). Dev F1 (final epoch) is on the development split of the *training* language(s); test F1 is on the test split of the language in the **Test** column. The first two rows share a dev value because they use the same EN-fine-tuned model evaluated on EN and DE test sets, respectively; the EN→DE row measures zero-shot cross-lingual transfer.

fine-tuned XLM-R” block). This barely moved the needle, lifting F1 from 87.10 to 87.24, still short of [Akbik et al. \(2018\)](#)’s 88.32. Fine-tuning XLM-R end-to-end on DE appears to have already saturated what could be extracted at the token level, leaving the CRF’s transition component little to learn.

This stands in contrast to [Souza et al. \(2019\)](#), who found that BERT + CRF on Portuguese NER outperformed the prior BiLSTM-CRF state of the art. The divergence is plausibly explained by the same saturation argument: XLM-R, after end-to-end fine-tuning on DE, may already capture the patterns of the CoNLL-2003 German test set well enough that the CRF has little remaining transition structure to add.

It helps to think about the two kinds of errors a CRF is best positioned to fix. The first is *illegal BIO transitions*—a stray I-PER after O or after a different entity type, which a token-level classifier can in principle produce because each token’s prediction is independent. The second is *span boundary errors* within multi-token entities, where a single entity is split across inconsistent labels rather than being grouped under one type. Of these, the first is exactly what the CRF’s transition matrix—especially with impossible transitions initialized to very negative values—rules out by construction at decoding time. The second is harder for a CRF to fix on its own: if XLM-R produces strong but inconsistent emissions across tokens of a multi-word entity, the CRF can only smooth this within the limits of the transition costs it has learned. The minimal effect of stacking a CRF on the fully fine-tuned XLM-R suggests that, after end-to-end fine-tuning, ill-formed BIO sequences are already rare enough that there is little for the CRF to clean up.

Configuration	F1
Flair + BiLSTM-CRF (Akbik et al., 2018)	88.32
XLM-R fine-tuned on DE (baseline)	87.10
<i>CRF on top of fine-tuned XLM-R</i>	
Top transformer layer unfrozen	87.24
<i>CRF + XLM-R jointly trained</i>	
Top 1 transformer layer unfrozen	80.96
Top 2 transformer layers unfrozen	83.24
Top 3 transformer layers unfrozen	83.59

Table 3: Effect of adding a CRF head to XLM-R fine-tuned on DE. F1 scores refer to the DE test set; the baseline is the DE-fine-tuned XLM-R from Table 2. In all rows below the baseline, all XLM-R encoder parameters are frozen except the top transformer layers explicitly marked as unfrozen; the classification layer is always trainable. The first row reports [Akbik et al. \(2018\)](#)’s prior state-of-the-art result on DE as an external reference point—a different architecture (Flair embeddings + BiLSTM-CRF), not an ablation of our setup.

We next asked whether jointly training the CRF with XLM-R from the pretrained checkpoint—where token-level signal has not yet been squeezed out—would let the CRF contribute more. Unfreezing only the top transformer layer ($k = 1$) gave 80.96 F1; increasing k to 2 and 3 raised this to 83.24 and 83.59 respectively. Performance plateaus in the low-to-mid 83s, well short of the 87–88 range achieved by fully fine-tuned XLM-R, suggesting that this restricted joint setup recovers some but not all of what end-to-end fine-tuning provides.

5.3 Does the CRF Help When Capacity Is Limited?

The plateaued joint-training results raise a natural question: in those experiments, was the CRF actually contributing, or was XLM-R itself doing all of the work? To isolate the CRF’s effect, we ran a controlled head-to-head experiment, holding the architecture and hyperparameters fixed and varying only the presence of the CRF head (Table 4). With just the top transformer layer unfrozen, XLM-R alone reaches 75.99 F1 on the DE test set. However, adding a jointly-trained CRF head lifts this to 80.53 F1. This ~4.5-point gain shows that when XLM-R is not already capturing all token-level structure—as it is in the fully fine-tuned baseline—learning explicit BIO transitions gives the model real additional traction.

This is the cleanest test of the CRF’s contribution we report, since architecture and hyperparameters

Configuration	F1
<i>Top transformer layer unfrozen</i>	
Fine-tuned on DE	75.99
Jointly trained with CRF head	80.53

Table 4: Head-to-head comparison under tight capacity: with only the top XLM-R transformer layer unfrozen (the classifier is always trainable), does adding a jointly-trained CRF head help? It does!

are held fixed and only the structured-output head varies. The interpretation aligns with the qualitative analysis above: in the capacity-limited regime, XLM-R’s per-token predictions are inevitably noisier and more prone to producing ill-formed BIO sequences, which is exactly where a CRF’s transition matrix can help. As the encoder becomes stronger (more layers unfrozen, longer training, larger model), its token-level predictions absorb increasingly more of the structure that a CRF would otherwise enforce, and the CRF’s marginal value shrinks.

Taken together, the three CRF experiments paint a coherent picture. The CRF’s contribution scales inversely with how much structural signal XLM-R has already absorbed: small when XLM-R is fully fine-tuned end-to-end (87.10 $\rightarrow$ 87.24), partial when XLM-R is jointly trained with several transformer layers unfrozen (the 80–84 plateau), and substantial when XLM-R’s capacity is sharply restricted (75.99 $\rightarrow$ 80.53). The CRF is therefore not a free improvement on top of any sufficiently strong encoder; it is a complement to the encoder, most useful when the encoder is leaving structural patterns on the table for it to capture.

6 Conclusion

We set out to replicate [Conneau et al. \(2020\)](#)’s XLM-R fine-tuning results on CoNLL-2003 NER and to test whether a CRF head could push XLM-R past [Akbik et al. \(2018\)](#) on German. The replication was largely successful: the XLM-R Base numbers match the published EN result and exceed the published DE result by 1.3 F1—a gain we attribute to having corrected the well-known encoding errors in the original German release. Concatenating English and German training data, by contrast, hurt rather than helped, most plausibly because the EN component biases the model toward an English-style “capitalized $\Rightarrow$ entity” prior that misfires on German common nouns. The CRF result is the more nuanced finding. Adding a CRF

on top of an already DE-fine-tuned XLM-R yields only a marginal gain (87.10 $\rightarrow$ 87.24 F1), still short of [Akbik et al. \(2018\)](#)’s 88.32, because end-to-end fine-tuning has already saturated the token-level signal that a CRF could exploit. But in a controlled, capacity-limited head-to-head, adding a jointly-trained CRF to XLM-R lifts F1 from 75.99 to 80.53—a clear demonstration that the CRF’s tag-transition modeling does have something to contribute when XLM-R is not already covering it on its own.

Two natural next steps suggest themselves. First, scale this joint XLM-R + CRF setup to a larger training corpus: the current results suggest that the CRF adds least when XLM-R has been allowed to absorb enough data to learn token-level patterns on its own, and whether its explicit transition structure could continue to contribute on top of a more strongly fine-tuned XLM-R—given enough data to expose patterns that XLM-R’s token-level predictions miss—is an open question worth investigating. Second, run the same setup with XLM-R Large rather than Base. Model scale was a major driver of [Conneau et al. \(2020\)](#)’s original gains, and a larger encoder may either further suppress the CRF’s contribution (by capturing even more token-level structure on its own) or expose new cases where transition modeling adds value.

References

- Alan Akbik, Duncan Blythe, and Roland Vollgraf. 2018. [Contextual string embeddings for sequence labeling](#). In *Proceedings of the 27th International Conference on Computational Linguistics*, pages 1638–1649.
- Alexis Conneau, Kartikay Khandelwal, Naman Goyal, Vishrav Chaudhary, Guillaume Wenzek, Francisco Guzmán, Edouard Grave, Myle Ott, Luke Zettlemoyer, and Veselin Stoyanov. 2020. [Unsupervised cross-lingual representation learning at scale](#). In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 8440–8451.
- Alexis Conneau and Guillaume Lample. 2019. [Cross-lingual language model pretraining](#). In *Advances in Neural Information Processing Systems 32 (NeurIPS)*.
- Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. [BERT: Pre-training of deep bidirectional transformers for language understanding](#). In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT)*, pages 4171–4186.

John Lafferty, Andrew McCallum, and Fernando C. N. Pereira. 2001. Conditional random fields: Probabilistic models for segmenting and labeling sequence data. In *Proceedings of the 18th International Conference on Machine Learning (ICML)*, pages 282–289.

Fábio Souza, Rodrigo Nogueira, and Roberto Lotufo. 2019. [Portuguese named entity recognition using BERT-CRF](#). *Preprint*, arXiv:1909.10649.

Erik F. Tjong Kim Sang. 2002. [Introduction to the CoNLL-2002 shared task: Language-independent named entity recognition](#). In *Proceedings of the 6th Conference on Natural Language Learning (CoNLL-2002)*.

Erik F. Tjong Kim Sang and Fien De Meulder. 2003. [Introduction to the CoNLL-2003 shared task: Language-independent named entity recognition](#). In *Proceedings of the Seventh Conference on Natural Language Learning at HLT-NAACL 2003*, pages 142–147.

A Hyperparameter Details

All training runs use random seed 1 and 3 epochs.

XLM-R fine-tuning (Table 2; baseline row of Table 3). AdamW with $\text{lr} = 2 \times 10^{-5}$ and weight decay 0.01.

CRF on top of fine-tuned XLM-R (Table 3, “CRF on top of fine-tuned XLM-R” block). Top transformer layer unfrozen; AdamW with $\text{lr} = 10^{-3}$.

CRF jointly trained with XLM-R, top- k transformer layers unfrozen (Table 3, “CRF + XLM-R jointly trained” block). AdamW with $\text{lr} = 10^{-4}$ for the XLM-R parameters, $\text{lr} = 10^{-3}$ for the CRF parameters, and weight decay 0 for both groups.

Top transformer layer + CRF head-to-head (Table 4). AdamW with $\text{lr} = 2 \times 10^{-5}$ for the unfrozen top transformer layer and the classification layer (weight decay 0.01 applied to the classifier), and $\text{lr} = 10^{-3}$ for the CRF parameters.